Student Workbook

One chapter per week: ideas, pictures, exercises and new words

> **Tip: How to use this workbook**
>
> Bring this workbook to every session. Read the chapter with your partner, try every exercise, and fill in the reflection at the end. It is fine to write in it, draw in it and make mistakes in it!

Contents

Week 1: Welcome Back, Python 2
Week 2: Functions That Give Back 6
Week 3: Lists: Collections of Things 9
Week 4: While Loops and Game Loops 12
Week 5: Dictionaries and Secret Codes 16
Week 6: Review and Checkpoint 20
Week 7: Files: Saving Your Stuff 24
Week 8: Testing: Prove It Works 28
Week 9: Event-Driven Games 32
Week 10: Project Studio 1 36
Week 11: Project Studio 2 41
Week 12: Showcase and Celebration 45

Week 1 Level 2: Intermediate

Welcome Back, Python

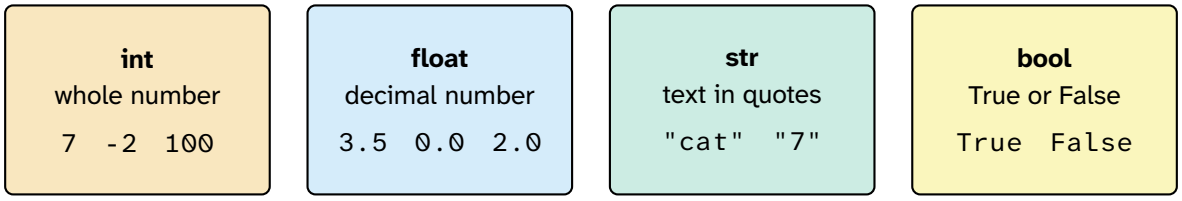
Every value has a type, and f-strings glue values into friendly messages.

What you will learn

- The four basic **types** of value: whole numbers, decimal numbers, text and True/False.
- How to build messages with **f-strings**.
- How to turn typed answers into numbers with `int()`.
- How to make smarter choices with `and`, `or` and `not`.

The big idea: every value has a type

Python sorts every value into a **type**. The type decides what Python can do with it. Here are the four types you will use most:



Quotes are the big clue. 7 is a number, but `"7"` is text that happens to show a 7. You can ask Python for the type of anything. Type this in the Thonny **Shell** (after >>>):

Try it in the Shell

```
type(7)        # <class 'int'>
type("7")      # <class 'str'>
type(3.5)      # <class 'float'>
type(True)     # <class 'bool'>
```

Bug hunt: text plus a number

Try `"5" + 5` in the Shell. Python shows a red **error message**. Always read it from the **bottom line up**:

Output

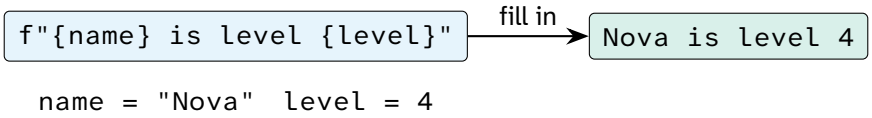
```
TypeError: can only concatenate str (not "int") to str
```

"Concatenate" means "glue together". Python can glue text to text, but not text to a number. This matters because **`input()` always gives back text**, even when you type a number. Turn it into a whole number with `int()`:

```
level = int(input("Level? "))      # the player types 4
print(level + 1)                   # 5
```

f-strings: fill in the blanks

Put an `f` in front of the quotes, and Python fills every pair of curly brackets with a value. It works with any type, so you never have to worry about text plus a number.



Smarter choices: and, or, not

Word	True when...	Game example
`and`	**both** sides are true	`has_key and level >= 3`
`or`	**at least one** side is true	`level >= 10 or has_map`
`not`	the opposite	`not game_over`

Each side needs a full comparison: write `energy >= 5 or energy == 1`, not `energy >= 5 or 1`.

Tip: Hero names, not real names

Your real name, school, address and birthday are **personal information**. Keep them private. In our programs you always type a made-up **hero name** or nickname.

Exercise 1.1: Hero Card

Folder: `exercises/week01/ex-hero-card`. Open `starter.py` and press **Run** (F5).

1. It crashes! Read the error from the bottom up. Write the last line here:

   _____

2. TODO 1: turn the level into a number with `int()`. Run again.
3. TODO 2–5: add the missing lines with f-strings: power, power points, health (`level * 12.5`) and "Legend?".
4. Which type is each value? Write int, float, str or bool:

   `hero_name` _____ `power_points` _____ `health` _____ `is_legend` _____

5. Check your work: open `check_my_work.py` and press **Run**.

Exercise 1.2: Mood Meter

Folder: `exercises/week01/ex-mood-meter`. The program asks for your energy (0–10) and whether you had a snack, then draws a meter. Finish `mood_for()`:

- "Super charged": energy 8 or more **and** a snack.
- "Ready to play": energy 5 or more **or** a snack.
- "Recharge time": everything else.

Before you run it, **predict** the mood. Then run and check.

energy	snack?	my guess	the program said
9	yes		
9	no		
2	yes		
3	no		

Last step: use `if not had_snack:` to print a friendly tip. Then check your work with `check_my_work.py`.

Vocabulary box

type The kind of value: int, float, str or bool. It decides what Python can do with it.

int A whole number, like 7 or -2.

float A number with a decimal point, like 3.5 or 2.0.

str (string) Text inside quotes, like `"cat"` or `"7"`.

bool True or False. Comparisons like `level >= 3` give a bool.

f-string Text with an f in front. Curly brackets inside are filled with values.

TypeError An error that means “these two types do not go together”.

personal information Facts that say who you really are or where to find you. Keep it private.

Reflection

Which value card or error surprised you today? Why?

Make up a game rule that uses `and` or `or`:

Week 2 Level 2: Intermediate

Functions That Give Back

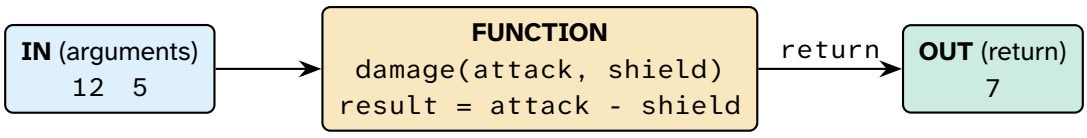
return hands an answer back; parameters make one function work for many jobs.

What you will learn

- How **parameters** let one function do many different jobs.
- How `return` hands an answer back, so your program can use it.
- Why printing an answer is not the same as returning it.
- How to write a **docstring** that explains your function.

The big idea: a function is a machine

A function is like a machine. Values go **in** through the parameters. The machine works on them. An answer comes **back out** with `return`.



A function with two parameters that returns an answer

```
def damage(attack, shield):
    """Return the damage a hit does."""
    return attack - shield

hit = damage(12, 5)       # 12 goes into attack, 5 goes into shield
print(f"Ouch! {hit} damage")
```

The values in the brackets when you *call* the function (`12` and `5`) are **arguments**. They fill the **parameters** (`attack` and `shield`) in the same order.

Print or return?

`print(answer)`	`return answer`
Shows the answer on the screen.	Hands the answer back to the code that called the function.
Like shouting it out of a window: you cannot keep it.	Like putting it in your hand: you can store it or do math with it.
The function gives back `None` (nothing).	The function gives back the answer.

Bug hunt clue: if you see the word `None` where a number should be, a function probably *printed* instead of *returning*.

Docstrings: notes for future-you

A **docstring** is a sentence in triple quotes just under the `def` line. It says what the function gives back. Thonny shows it when you hover over the function name, and your friends can read it to understand your code.

What happens in a function stays in a function

Variables you make inside a function (like `result`) only exist inside it. Outside the function, Python says `NameError`. If you need the value outside, `return` it.

Exercise 2.1: Game Math

Folder: `exercises/week02/ex-game-math`. Open `starter.py` and press **Run**.

1. Find the word `None` in the output. Which function caused it? _____
2. TODO 1: change `print(...)` to `return ...` in `xp_needed()`.
3. Predict first, then finish `damage()` (TODO 2 and 3): `damage(12, 5)` = _____ `damage(3, 5)` = _____
4. TODO 4: finish `is_critical()`. Tip: `return roll >= 19` gives back True or False by itself.
5. TODO 5: remove the # from the two health lines. Health left: _____
6. Check your work with `check_my_work.py`.

Exercise 2.2: Flower Garden

Folder: `exercises/week02/ex-flower-garden`. Press **Run**: all three flowers land in the same spot with 4 petals.

1. TODO 1: `petal_turn(petals)` should return `360 / petals`. How far does it turn for 6 petals? _____
2. TODO 2 and 3: use `t.goto(x, y)` so each flower appears where it was asked to.
3. TODO 4: repeat once for each petal: `range(petals)`.
4. Fill in the arguments you used for your favorite flower:

   `draw_flower(t, x =` _____, y = _____, petals = _____, color = _____`)`

Vocabulary box

parameter A named slot in a function's `def` line, like `attack`. It receives a value.

argument The value you hand in when you call the function, like `12`.

return Hands an answer back to the code that called the function, and ends the function.

None Python's word for "nothing". A function with no `return` gives back `None`.

docstring A sentence in triple quotes under `def` that explains what the function does.

scope Where a variable can be seen. Variables made inside a function stay inside it.

Reflection

Explain print versus return to someone who missed today (use your own example):

Write the docstring you are most proud of:

Week 3 Level 2: Intermediate

Lists: Collections of Things

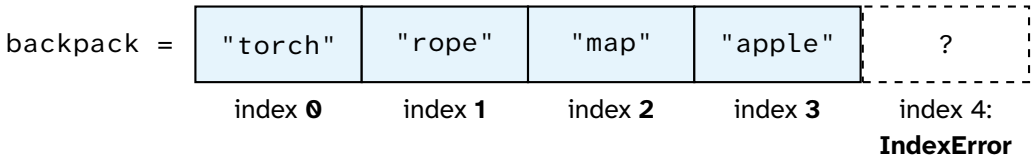
A list keeps many values in order, and the first one is number 0.

What you will learn

- How to keep many values in one **list**.
- How to find an item by its **index**, starting from 0.
- How to add items with `append` and count them with `len`.
- How to visit every item with a `for` loop.
- How to pick colors that work for everyone.

The big idea: a row of lockers

A list is like a row of lockers. Each locker has a number painted on it, called its **index**. The first locker is number **0**.



Making and using a list

```
backpack = ["torch", "rope", "map"]
backpack.append("apple")    # adds to the END of the list
print(len(backpack))        # 4   how many items
print(backpack[0])          # torch   the first item
print(backpack[-1])         # apple   -1 means the last item
```

With 4 items, the index numbers are 0, 1, 2 and 3. Asking for `backpack[4]` crashes with `IndexError: list index out of range`. This is called an **off-by-one** bug.

Two ways to loop over a list

Visit every item	Visit every index number
`for item in backpack:` `    print(item)`	`for i in range(len(backpack)):` `    print(i, backpack[i])`
Use it when you only need the items.	Use it when you also need the position number.

Colors for everyone

Some people see colors differently, often red and green. These eight **Okabe-Ito** colors stay different for almost everyone. Use them, and always add a label, number or shape too, so color is never the only clue.

■	orange	`#E69F00`	■	blue	`#0072B2`
■	sky blue	`#56B4E9`	■	vermillion	`#D55E00`
■	bluish green	`#009E73`	■	reddish purple	`#CC79A7`
■	yellow	`#F0E442`	■	black	`#000000`

Exercise 3.1: Palette Stripes

Folder: `exercises/week03/ex-palette-stripes`. Press **Run**: only one stripe appears.

1. TODO 1: add blue and vermillion with `palette.append(...)`.
2. TODO 3: draw a stripe for every color with `for i in range(len(colors)):`. Label each stripe `str(i)` and move `x` along.
3. TODO 4: print the first color (`palette[0]`) and the last color (`palette[-1]`).
4. My palette has _____ colors, so the labels go from `0` to _____.

Exercise 3.2: Adventure Inventory

Folder: `exercises/week03/ex-inventory`. Press **Run**. It crashes!

1. Read the error from the bottom up. Error name: _____ In which function? _____
2. Draw the backpack `["torch", "rope", "map", "apple"]` as boxes with their index numbers:

3. TODO 3: fix `last_item()`. TODO 1: `append` in `add_item()`. TODO 2: loop in `has_item()`.
4. Check your work with `check_my_work.py`.

Exercise 3.3: Top 3 High Scores

Folder: `exercises/week03/ex-high-scores`. Two new tricks: `sorted(scores, reverse=True)` makes a new list, biggest first, and `best[:3]` takes the first three items.

1. Predict: the top 3 of `[120, 45, 300, 80]` are _____
2. TODO 1: sort, then take three. TODO 2: print `1st: 300`, `2nd: 120`, `3rd: 80` using the `places` list.

Vocabulary box

list Many values kept in order inside square brackets, like `["torch", "rope"]`.

item One value inside a list.

index The position number of an item. The first item has index `0`.

append Add an item to the end of a list: `backpack.append("map")`.

len Counts the items in a list: `len(backpack)`.

IndexError The error you get when you ask for an index that does not exist.

off-by-one A bug where a number is one too big or one too small, like using `len` as the last index.

accessible Designed so that everyone can use it, including people who see colors differently.

Reflection

With a list of 10 items, what is the index of the last item? Why?

Describe a list you could use in a game you would like to make:

Week 4 Level 2: Intermediate

While Loops and Game Loops

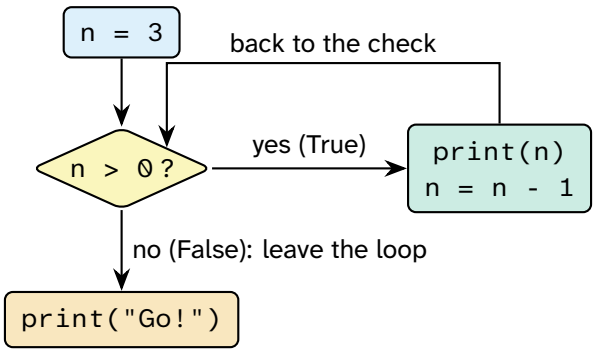
A while loop repeats until something changes, and every game is a loop.

What you will learn

- How a **while loop** keeps going until something changes.
- How to spot and stop an **infinite loop**.
- How to keep asking until the player types a good answer (**input validation**).
- How a **loop inside a loop** turns a list of strings into pixel art.

The big idea: repeat until something changes

A `for` loop knows how many times it will go round. A `while` loop does not: it checks a condition at the top, and goes round again *while* the condition is `True`.



This prints 3, 2, 1, Go!. The line `n = n - 1` is the **change**: without it, n stays 3, the check is always `True`, and the loop never ends. That is an **infinite loop**.

Tip: Stuck in a loop?

Press the red **Stop** button in Thonny's toolbar. Then ask two questions: **What is checked?** and **What changes inside the loop, so the check can become False?**

while True and break

Sometimes it is easier to loop "for ever" and jump out when you are done. `break` leaves the loop straight away. Inside a function, `return` leaves the loop *and* the function.

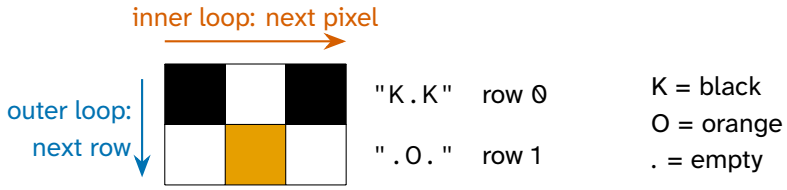
Keep asking until the answer is good

```
while True:
    answer = input("Type yes or no: ").lower()
    if answer == "yes" or answer == "no":
        break                       # good answer: leave the loop
    print("Please type yes or no.")
print("Thanks!")
```

Checking the player's answer like this is called **input validation**. Games do it all the time, so a typo never crashes them. `answer.isdigit()` is `True` only if the answer is made of the digits `0`–`9`: then `int(answer)` is safe.

A loop inside a loop: pixel art

A picture can be a list of strings: one string for each row, one letter for each pixel. The **outer loop** goes down the rows. The **inner loop** goes along one row, pixel by pixel.




```
for row in grid:            # outer loop: one row at a time
    for pixel in row:       # inner loop: one letter at a time
        print(pixel)
```

Exercise 4.1: Guess My Number

Folder: `exercises/week04/ex-guess-number`. Open `starter.py` and press **Run**.

1. Guess until you are right. Does the game stop? If not, press **Stop**. Why does it keep going?

---

2. TODO 1: add 1 to `tries` each time round. TODO 3: `break` when the result is `"Correct"`.
3. Now type `banana`. Crash! TODO 2: make `ask_number()` keep asking until the answer is a

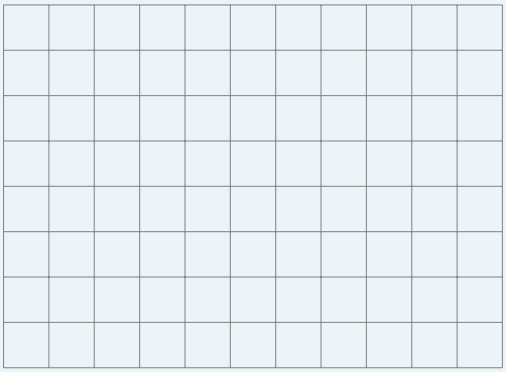
whole number from `low` to `high`.

4. Be the tester: try `banana`, `99`, `0` and `-3`. Does your game survive them all? ☐
5. Check your work with `check_my_work.py`.

Exercise 4.2: Pixel Art

Folder: `exercises/week04/ex-pixel-art`. Only the top row of the space invader appears.

1. TODO 1: wrap the drawing in an outer loop `for row in grid:`. Move down after each row: `y = y - size`.
2. TODO 2: finish `count_pixels()` with a loop inside a loop.
3. Design your own picture (8 rows of 11 letters fit nicely). Plan it here first:



Check your work with `check_my_work.py`.

Vocabulary box

while loop A loop that goes round again and again while its condition is True.

condition The check at the top of a while loop, like `n > 0`. It is True or False.

infinite loop A loop that never stops, because nothing changes the condition. Press Stop!

break Jumps out of a loop straight away.

input validation Checking the player's answer and asking again if it is not OK.

nested loop A loop inside another loop, like rows and pixels.

game loop The loop at the heart of a game: get input, update, show, repeat.

Reflection

When would you use a `while` loop instead of a `for` loop?

Name one way a game could be unfair to players. How would you make it fair?

Week 5 Level 2: Intermediate

Dictionaries and Secret Codes

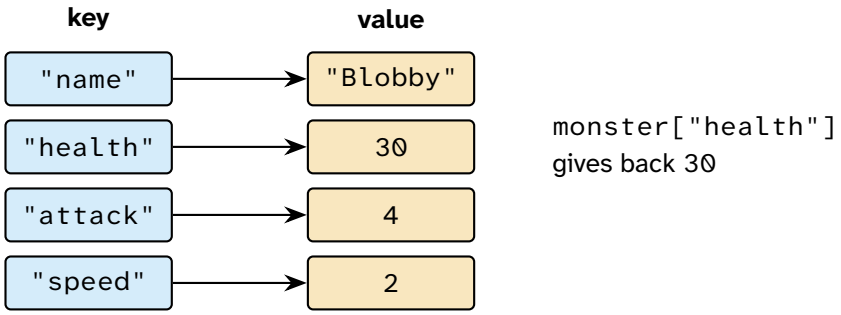
A dictionary looks things up by name, not by number, and can even hold a secret code.

What you will learn

- How a **dictionary** stores values under names called **keys**.
- How to change values, add new keys and loop over everything with `.items()`.
- How to encode and decode secret messages with a **Caesar cipher**.
- What a `KeyError` means, and why real passwords must stay secret.

The big idea: look things up by name

A list finds things by number: `backpack[0]`. A dictionary finds things by name. Each name is a **key**, and the thing it gives back is its **value**. Dictionaries use curly brackets, with a colon between key and value.



A monster as a dictionary

```
monster = {"name": "Blobby", "health": 30, "attack": 4, "speed": 2}
print(monster["health"])          # 30
monster["health"] = 40            # change a value
monster["color"] = "green"        # add a new key
for key, value in monster.items():
    print(f"{key}: {value}")      # every key with its value
```

Bug hunt: KeyError

If you look up a key that is not there, Python stops with a `KeyError`:

Output

```
KeyError: 'magic'
```

The last line shows the key Python could not find. Check the spelling: `"Health"` and `"health"` are different keys! To be safe, check first: `if "magic" in monster:`

Secret codes: the Caesar cipher

Julius Caesar sent secret messages by moving every letter a few places along the alphabet. With a **shift of 3**:

plain: a b c d e f g h i j k l m
secret: **d e f g h i j k l m n o p**

plain: n o p q r s t u v w x y z
secret: **q r s t u v w x y z a b c**

So `hi` becomes `kl`. At the end the alphabet wraps round: x, y, z become a, b, c. To **decode**, shift the other way. That table is a lookup, so in Python it is a dictionary: `{"a": "d", "b": "e", ...}`.

Tip: Real secrets need real protection

A Caesar cipher is easy to crack: there are only 25 shifts to try! Real apps use much stronger **encryption**. Your real passwords stay between you and your parent or carer. Never type a real password into a program you or a friend wrote.

Exercise 5.1: Monster Stats

Folder: `exercises/week05/ex-monster-stats`. Open `starter.py` and press **Run**.

1. TODO 1: in `stat_report()`, loop with `for key, value in monster.items():` and build lines like `health: 30`.
2. TODO 2: `level_up()` adds 10 to health and 2 to attack.
3. Run it and type `magic`. Write the last line of the error:

   _____

4. TODO 3: fix `get_stat()` so a missing stat gives `0`. Then check your work with `check_my_`

`work.py`.

Exercise 5.2: Caesar Cipher

Folder: `exercises/week05/ex-caesar-cipher`. Run `starter.py` first and look at the dictionary it prints.

1. TODO 1: finish `encode()`. For each letter: if it is in the key, add `key[letter]`; otherwise add the letter itself.
2. TODO 2: finish `decode()`. Hint: shifting by `-shift` goes backwards.
3. Encode a kind message for your partner. Secret version: _____ shift: _____
4. Decode this one (shift 3) on paper or with your program: `zhoo grqh!` _____
5. Check your work with `check_my_work.py`.

Vocabulary box

dictionary A collection that stores values under names (keys), written with curly brackets.

key The name you look something up by, like `"health"`.

value What a key gives back, like `30`.

.items() Gives every key together with its value, so a loop can use both.

KeyError An error that means “there is no such key”. Check the spelling!

cipher A way of turning a message into a secret code.

encode / decode Turn a message into secret code / turn it back again.

encryption Strong secret codes that computers use to keep data safe.

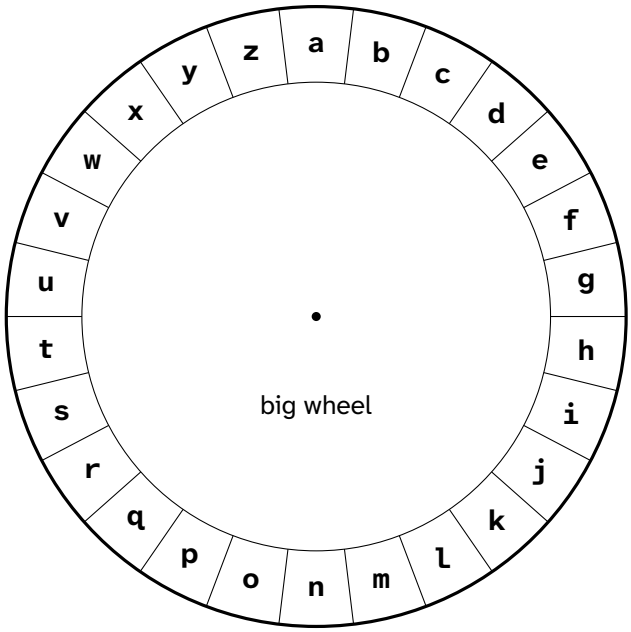
Reflection

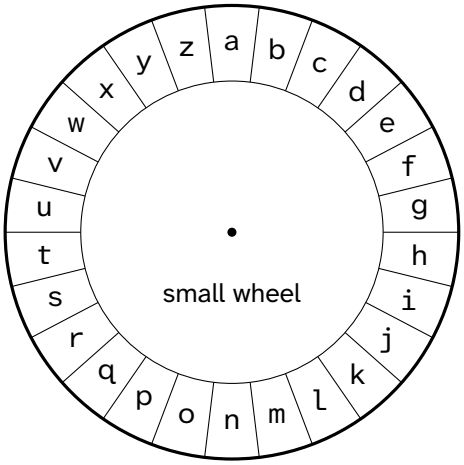
Give a game example where a dictionary is better than a list:

Why should a real password never go into a program or a message?

Cut-out: your Caesar cipher wheel

Cut out both wheels along the thick outer circles. Put the small wheel on top of the big one and push a split pin (or a pencil point) through both centers. Turn the small wheel so that small **a** sits under big **d**: that is shift 3. Encode: small wheel → big wheel. Decode: big wheel → small wheel.





Week 6 Level 2: Intermediate

Review and Checkpoint

Bugs are normal: detectives find them with a strategy, not by luck.

What you will learn

- How to read a **traceback** from the bottom up.
- Three more detective tools: **print-debugging**, the **rubber duck** and the **trace table**.
- How to write a short **bug report**, so you (and others) remember what you learned.

The big idea: debug like a detective

Every programmer writes bugs. Good programmers find them with a **strategy**: they collect clues, change **one** thing and test again.

Tool 1: read the traceback from the bottom up

When a program crashes, Thonny shows a red **traceback** like this one:

Output

```
Traceback (most recent call last):
  File "starter.py", line 24, in <module>
  File "starter.py", line 17, in make_card
  File "starter.py", line 10, in card_line
    return label + ": " + value
TypeError: can only concatenate str (not "int") to str
```

1. **WHAT** went wrong: the **last line**. Here: a `TypeError`, text + number.
2. **WHERE**: the `File` line just above it. Here: line 10, in the function `card_line`.
3. **HOW** it got there: the lines further up. The main program (line 24) called `make_card`, which called `card_line`.

In Thonny you can click the blue link to jump straight to that line.

Error name	It usually means…
`TypeError`	two types that do not go together, like text + number
`IndexError`	a list index that does not exist (remember: the last index is `len - 1`)
`KeyError`	a dictionary key that does not exist (check the spelling)
`NameError`	a variable name Python does not know (typo, or missing quotes)
`ValueError`	the right type but a bad value, like `int("banana")`

Tool 2: print-debugging

Not sure what is inside a variable? Ask Python! Add a `print()` line, run, look, then remove it.

```
print("best is", best, "length", len(best))
```

Tool 3: the rubber duck

Explain your code, line by line, out loud, to a partner, a toy or a rubber duck. Say what each line *should* do. Very often you hear yourself say the bug before you reach the end. This is the best tool for bugs that do not crash.

Tool 4: the trace table

Follow the code with a pencil, like a very slow computer. Write a column for each variable and a row each time round the loop. When the table shows a value you did not expect, you have found the bug.

Tip: Bugs are normal

If you are stuck, you are not bad at coding: you are in the middle of learning. Say “I can’t fix this *yet*”, pick a tool and try one thing.

Bug-hunt workshop

Each station has a folder in `exercises/week06`. Open `starter.py`, find the bug, fix it, then check with `check_my_work.py`. Fill in a bug report for each bug you fix.

Exercise 6.1: Station 1: Hero Card (TypeError)

Folder: `ex-bug-hero-card`. We do this one together.

Error name: _____ Line: _____ Tool I used: _____

I expected: _____ It really did: _____

What I changed:

_____

Exercise 6.2: Station 2: High Scores (IndexError, 2 bugs)

Folder: `ex-bug-high-scores`. Draw the list with its index numbers. Try `print("best is", best)` inside `podium()`.

Bug 1 – error name: _____ line: _____ what I changed: _____

Bug 2 – error name: _____ line: _____ what I changed: _____

Exercise 6.3: Station 3: Rocket Countdown (infinite loop)

Folder: `ex-bug-countdown`. The safety net stops the stuck loop. Fill in the trace table for `countdown(5)` as the starter is now:

round	n	numbers
1	5	[5]
2		
3		

What should change each time round, but does not? _____

What I changed:

_____

Exercise 6.4: Station 4: Loot Shop (3 sneaky bugs, no crash)

Folder: `ex-bug-loot-shop`. First work it out on paper for the basket `[12, 5, 30]`:

Total: _____ Cheapest: _____ Can you buy it all with `50` coins? _____

Now run it and explain each function to your duck. The three bugs I fixed:

_____

_____

Exercise 6.5: Station 5: Paper detectives (no computer)

This function should give back the biggest number in a list. Fill in the trace table for `biggest([-5, -2, -8])`.

```
def biggest(numbers):
    best = 0
    for n in numbers:
        if n > best:
            best = n
    return best
```

n	n > best?	best
start	–	0
-5		
-2		
-8		

It gives back _____, but the biggest number is _____. What is the bug, and how would you fix it?

Vocabulary box

bug A mistake in a program that makes it crash or give the wrong answer.

debugging Finding and fixing bugs, step by step, with a strategy.

traceback The red error message. Read it from the bottom up: what, then where.

print-debugging Adding `print()` lines to see what is really inside your variables.

rubber duck debugging Explaining your code line by line out loud, to find the bug yourself.

trace table A table where you follow code by hand and write each variable's value.

bug report A short note: what went wrong, where, and what you changed.

Reflection

Six weeks ago I could not … and now I can …

Which detective tool will you try first next time? Why?

Week 7 Level 2: Intermediate

Files: Saving Your Stuff

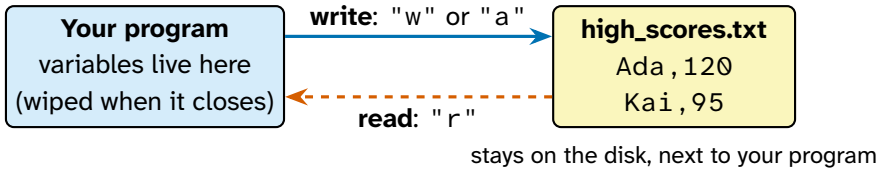
A program forgets everything when it closes, unless it writes things down in a file.

What you will learn

- How to **save** data in a text file, and how to **load** it again next time.
- The three ways to open a file: `"r"` read, `"w"` write, `"a"` append.
- How to clean up lines with `strip()` and cut them into pieces with `split()`.
- What to do when the file is missing, and what a game should never save about you.

The big idea: programs forget, files remember

When a program closes, all its variables are wiped. Your high score? Gone. A **file** is like a notebook in a drawer: it stays on the computer after the program closes, so next time the program can read it again.



Writing to a file

Save one score

```
with open("high_scores.txt", "a") as file:
    file.write("Ada,120\n")
```

- `open(name, mode)` opens the file. `with` closes it again for you when the indented block ends.
- `\n` means “new line”. Without it, everything ends up squashed on one line.

Mode	Means	Like a notebook…
`"r"`	read	Just read it. Crashes if the notebook is not there.
`"a"`	append (add to the end)	Write on the next empty line. Makes a new notebook if needed.
`"w"`	write (replace)	Rip out every page, then write. **Careful: old data is gone!**

Reading a file, line by line

Load the scores back

```
scores = []
with open("high_scores.txt", "r") as file:
    for line in file:
        line = line.strip()            # "Ada,120\n" -> "Ada,120"
        if line != "":                 # skip empty lines
            parts = line.split(",")    # "Ada,120"   -> ["Ada", "120"]
            scores.append([parts[0], int(parts[1])])
print(scores)                          # [['Ada', 120], ['Kai', 95]]
```

Everything you read from a file is **text** (a `str`), just like `input()`. Use `int()` to turn `"120"` into the number `120`.

Bug hunt: the file is not there!

The very first time a game runs, there is no score file yet. Reading it gives:

Output

```
FileNotFoundError: [Errno 2] No such file or directory: 'high_scores.txt'
```

Check the spelling of the name first. If the file really is new, catch the error with `try` and `except`:

```
try:
    with open("high_scores.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("No scores yet - you will be the first!")
```

`try` means “have a go”. `except FileNotFoundError` means “if the file is missing, do this instead of crashing”.

Exercise 7.1: Save High Scores

Folder: `exercises/week07/ex-save-high-scores`. Open `starter.py`, press **Run** twice. Does it remember your first game?

1. TODO 1: in `save_score()`, open the file with `"a"` and write one line `nickname,score`.
2. TODO 2: in `load_scores()`, strip, skip empty lines, split at the comma and append `[nickname, score]`.
3. TODO 3: if the file is missing, make an empty one with `"w"`.
4. Play three times. Open `high_scores.txt` (Finder: double-click it). Copy the first line here:

   _____

5. Check your work with `check_my_work.py`.

Exercise 7.2: Word Guess (choose this one or 7.3)

Folder: `exercises/week07/ex-word-guess`. The secret words live in `words.txt`, one per line.

1. TODO 1: `load_words()` reads every line, strips it, makes it lowercase with `lower()` and skips empty lines.
2. TODO 2: `show_progress()` shows guessed letters and _ for the rest, like `p _ _ e _`.
3. Play a round with your partner, then add three new words to `words.txt`. My new words:

   _____

Exercise 7.3: Dot Art from a File (choose this one or 7.2)

Folder: `exercises/week07/ex-dot-art-file`. Each line of `my_art.txt` is one dot: `x,y,size,color`.

1. TODO 1: `parse_line()` turns `"-150,-60,40,#009E73"` into `[-150, -60, 40, "#009E73"]`.
2. TODO 2: `load_dots()` reads every line that is not empty. Run it: a caterpillar under a sun!
3. TODO 3: `save_dots()` writes every dot back, one per line. Which mode is right here, and why?

   _____

4. Change `my_art.txt` to draw something new. Colors: #E69F00 #56B4E9 #009E73 #F0E442 #0072B2 #D55E00 #CC79A7.

Tip: What should a game save?

A file stays on the computer, and anyone who uses the computer can open it. Save only what the program needs: nicknames, scores, words, drawings. Never save real names, addresses, schools, birthdays or passwords.

Vocabulary box

file Data saved on the computer's disk. It is still there after the program closes.

open / with `open` gets a file ready to use; `with` closes it again automatically.

read / write / append The modes `"r"`, `"w"` and `"a"`: look, replace everything, or add to the end.

strip() Removes spaces and the invisible new-line `\n` from both ends of a string.

split(",") Cuts a string into a list at every comma: `"a,b"` becomes `["a", "b"]`.

FileNotFoundError The error you get when you try to read a file that does not exist.

data privacy Deciding what information about people is saved, who can see it, and keeping it safe.

Reflection

What is the difference between a variable and a file?

My partner's game saved "only the newest score". What do you think the bug was?

Name two things a game should never save about you.

Week 8 Level 2: Intermediate

Testing: Prove It Works

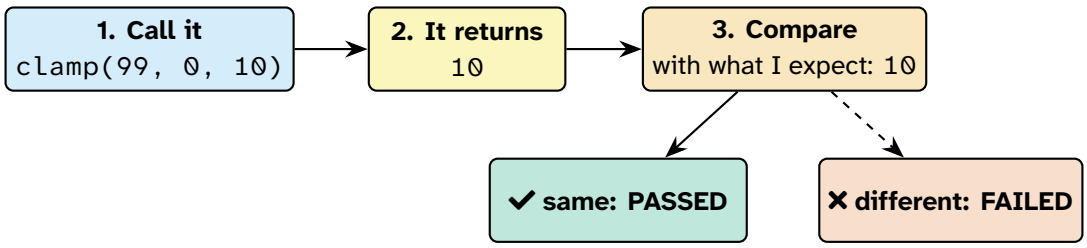
A test is a tiny program that checks your program, so bugs get caught before players find them.

What you will learn

- How to write a **test** with `assert`.
- How to run your tests with **pytest** and read PASSED and FAILED.
- How to choose **normal** and **edge** test cases that catch sneaky bugs.
- How to use failing tests as a map to find and fix bugs.

The big idea: let the computer check your code

Checking a game by hand is slow: run it, play, look at the score... again and again. A **test** is a tiny function that checks another function for you, in less than a second. It uses the word `assert`, which means "I am sure this is true".



A test for clamp(), which keeps a number between two limits

```
from clamp_tools import clamp

def test_too_big_comes_down_to_high():
    assert clamp(99, 0, 10) == 10, "99 is too big, so it should become 10"
```

A test has three parts: **call** the function with values you choose, say what you **expect** with ==, and add a **message** for when it goes wrong. The function's name must start with `test_`, so that pytest can find it.

Running your tests

- **Easy way:** open your test file and press **Run**. The bottom of the file starts pytest for you.
- **Programmer way: Tools > Open system shell...**, type `python -m pytest starter.py -v` and press `Return` (if it says "command not found", use `python3`).

Output

```
starter.py::test_middle_value_stays_the_same PASSED     [ 50%]
starter.py::test_too_small_goes_up_to_low FAILED        [100%]
E       AssertionError: -3 is too small, so it should become 0
E       assert 0 == 5
E        +  where 0 = clamp(-3, 0, 10)
============ 1 failed, 1 passed in 0.03s ============
```

Read it from the bottom up. The last line is the score. The `E` lines say what went wrong: `clamp` gave back `0` but the test expected 5. When a test fails, ask: **is the code wrong, or is my expected answer wrong?**

Normal and edge cases

A **normal case** is what usually happens. An **edge case** is a strange corner: that is where bugs love to hide.

Function	Normal case	Edge cases
`clamp(value, 0, 10)`	`clamp(5, 0, 10)` gives 5	too small, too big, exactly `0`, exactly `10`
`best_score(scores)`	best in the middle	empty list, one score, best is first, best is last
`encode(msg, shift)`	`"abc"` with shift 1	letters near z, spaces and `!`, shift `0`

Exercise 8.1: Test the Clamp

Folder: `exercises/week08/ex-test-the-clamp`. `starter.py` is **your test file**. Read `clamp_tools.py` first.

1. Press **Run**. How many tests say PASSED? _____ Why do the empty ones pass?

   _____

2. TODO 1 and TODO 2: write an `assert` line in each empty test. TODO 3: add a test for a value exactly on a limit.

3. Change one expected answer to something wrong. Run. Copy the last line of the output:

4. Change it back. Then run `check_my_work.py`: it tries your tests on three **broken** clamps. Catch them all!

Exercise 8.2: Test the Best Score

Folder: `exercises/week08/ex-test-best-score`. `best_score(scores)` finds the highest score. Plan your tests first:

I call...	I expect...	Normal or edge?
`best_score([3, 9, 2])`	`9`	normal
`best_score([])`		

Write the tests in `starter.py`, press **Run**, then check with `check_my_work.py`.

Exercise 8.3: Fix the Bugs the Tests Found

Folder: `exercises/week08/ex-fix-the-bugs`. Four game helpers, one bug each. The tests are already written.

1. Run `check_my_work.py`. Pick one `TRY AGAIN` and read its hint.
2. Find that test in `test_fix_the_bugs.py`. Read it like a sentence.
3. Fix **one** function in `starter.py`. Never change the test! Run the checks again. Repeat.

Fixed: ☐ `lives_after_hit` ☐ `medal` ☐ `name_tag` ☐ `average_score`

Tip: Why testing matters

In 1999 a spacecraft was lost on its way to Mars because two parts of its software used different units, and no test checked them together. People can only trust apps and games that have been tested. Testing your game is a way of being fair to your players.

Vocabulary box

test A small function that checks another function and says PASSED or FAILED.

assert "I am sure this is true." If it is false, Python stops with `AssertionError`.

pytest A helper that finds every function called `test_...`, runs it and reports the results.

normal case An everyday input, like a list of three scores.

edge case A strange corner: empty, zero, the first, the last, exactly on a limit.

expected vs actual What the test says should happen, and what really happened.

Reflection

Think of a game you like. Write one edge case its programmers should test.

When a test fails, what two things could be wrong?

Which was harder: writing tests or fixing bugs? Why?

Week 9 Level 2: Intermediate

Event-Driven Games

Keys and timers make things happen; a game loop moves the world one frame at a time.

What you will learn

- How to make keys call your functions with `onkey` and `listen`.
- How a **game loop** with `ontimer` moves everything one frame at a time.
- How to keep the game's **state** in a dictionary and check if two things are **touching**.
- How to start planning your own final project.

The big idea: games wait for events

Most programs you wrote ran from top to bottom and stopped. A game **waits**. When something happens (a key press, a timer ringing), turtle calls one of your functions. The thing that happens is an **event**; the function that answers it is a **handler**.

Keys call handlers

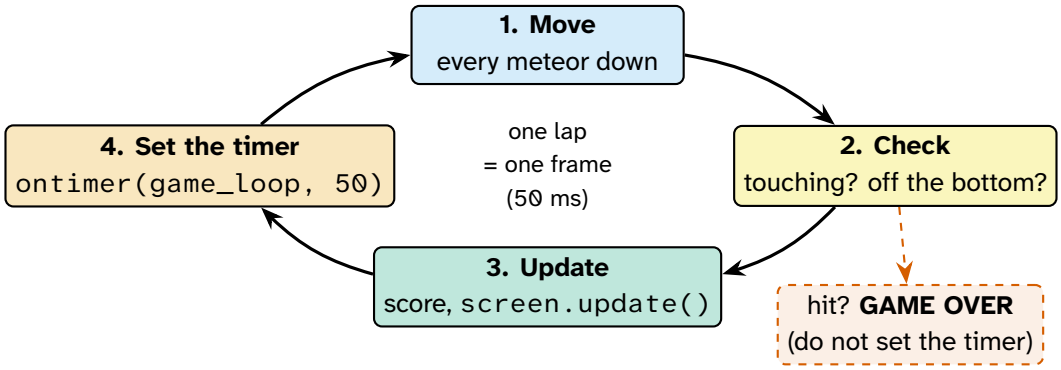
```
def go_left():
    player.setx(player.xcor() - 20)

screen.onkey(go_left, "Left")   # when Left is pressed, call go_left
screen.listen()                 # switch on the "doorbell" for keys
```

Notice: `go_left` has **no brackets** in the `onkey` line. We hand turtle the function, and turtle calls it later. With brackets, Python would call it right away, only once.

The game loop

A game is like a flip book: every **frame** the world moves a tiny bit. Our game loop does one frame, then asks turtle to call it again soon.



One frame of Dodge the Meteors (shortened)

```
state = {"score": 0, "running": True}

def game_loop():
    for meteor in meteors:                      # a list of turtles
        meteor.sety(meteor.ycor() - 8)          # 1. move
        if is_touching(meteor.xcor(), meteor.ycor(),
                       player.xcor(), player.ycor(), 30):
            state["running"] = False            # 2. check: hit!
    if state["running"]:
        screen.ontimer(game_loop, 50)           # 4. again in 50 ms
```

Touching = close enough

Two things touch when the distance between them is small. `math.dist` measures the distance between two points: `math.dist((0, 0), (30, 40))` gives `50.0`. If that is less than about 30, the meteor hit the player.

Exercise 9.1: Etch-a-Sketch

Folder: `exercises/week09/ex-etch-a-sketch`. Run `starter.py`, **click the window**, press the Up arrow.

1. TODO 1: add `screen.listen()`. Now Up draws a line.
2. TODO 2: write `go_down()`, `go_left()`, `go_right()` (headings 270, 180, 0).
3. TODO 3: connect the keys: `"Down"`, `"Left"`, `"Right"`, `"space"`, `"n"`, `"c"`.
4. TODO 4: `c` clears the drawing. Draw something and describe it here:

Exercise 9.2: Dodge the Meteors

Folder: `exercises/week09/ex-dodge-meteors`. You are the blue turtle; the round orange-red meteors fall.

1. TODO 1: `move_player()` returns the new x, using `clamp()` so you stop at the walls.
2. TODO 2: `is_touching()` uses `math.dist` and <.
3. TODO 3: connect `"Left"` and `"Right"`, then `listen()`.
4. TODO 4: at the end of `game_loop()`, set the timer again. Now play! My best score: _____
5. Which functions are **logic** (no turtle) and which are **drawing**? Why is it useful to keep them apart?

Your final project starts now!

For the next three weeks you will build your own program, alone or with a partner. Read the **Project Brief** and choose:

Option A: Arcade Game	Option B: Art Studio
A turtle game with keys, a timer and a score, like Dodge the Meteors or Star Catcher. The file saves high scores.	A program that makes designs (often random) and can save and load them, like Dot Art from a File.

Both must use: functions that return values, a list, a dictionary, a loop or game loop, a file, and at least 3 tests. **Milestone M1 (start of next week):** Planning page 1 (pitch), page 2 (pseudocode or flowchart) and, in a pair, page 3 (task split).

Tip: Borrowing pictures or sounds

Pictures and sounds belong to the people who made them. Only use ones with an open license (CC0: free for anyone; CC BY: free if you give credit), ask a grown-up to help you find them, and write a credit in your code, like `# Star sound by Mei (CC BY)`. Turtle's own shapes are always fine!

Vocabulary box

event Something that happens while a program runs: a key press, a click, a timer ringing.

handler The function that runs when an event happens.

listen() Tells the turtle window to start hearing key presses.

game loop A function that does one frame of the game, then sets a timer to run again.

frame One tiny step of a game: everything moves a little, then it is drawn.

collision Two things touching. We check it with the distance between them.

license / attribution The rules for using someone's work / giving them credit for it.

🖌 Reflection

What happens if you forget `ontimer` at the end of the game loop?

Explain to a friend why `onkey(go_left(), "Left")` does not work.

My project idea in one sentence:

Week 10 Level 2: Intermediate

Project Studio 1

Plan first, then build the core: a small program that works beats a big one that does not.

What you will learn

- How to turn your project plan into pseudocode and a flowchart.
- How to split your project into functions, and split the work fairly with your partner.
- How to build the **core loop** of your project: the smallest version that runs (Milestone M2).
- How to review a partner's code kindly and usefully.

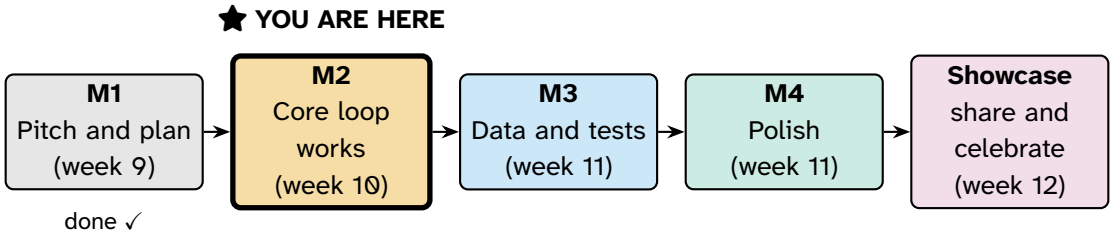
The big idea: build a skateboard first

If you want to build a car, you do not start with one perfect wheel. Nobody can ride a wheel! You build a skateboard first: tiny, but you can ride it. Then a scooter, a bike, a car. Every version works.

Your project works the same way. The **core loop** is your skateboard:

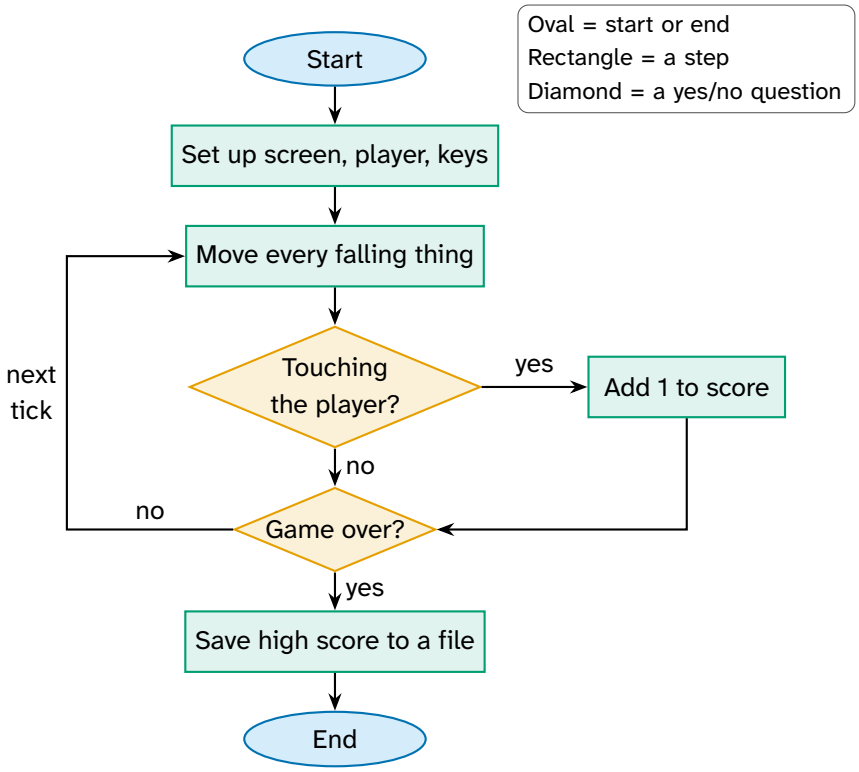
- **Arcade Game:** a window, a player that moves with the keys, and a timer loop that keeps running (`screen.ontimer`).
- **Art Studio:** a `while` loop that asks what to do (draw, save, load, quit) and does it.

No score, no title screen, no fancy colors yet. Those come later, and they are easy once the skeleton walks.



Pseudocode and flowcharts

Pseudocode is your program written in plain English, one step per line, before you write Python. A **flowchart** draws the same steps as shapes joined by arrows. Three shapes are all you need: **ovals** for start and end, **rectangles** for steps, and **diamonds** for yes/no questions. Here is the flowchart for a catching game:



The same plan as pseudocode:

Pseudocode

```
set up the screen, the player and the keys
repeat every tick:
    move every falling thing down
    if a falling thing touches the player: add 1 to the score
    if the game is over: save the high score and stop
```

Every line that does one job can become a **function**. “Move the player but not off the screen” becomes `next_x(x, direction)`: the position and direction go in, the new position comes out. That is **decomposition**: breaking a big problem into small named parts.

Exercise 10.1: Paper prototype and plan check (M1)

1. Draw your screen on paper. Use coins for things that move.
2. One partner is “the computer”, the other is the player. Play for one minute: every clap is one tick.
3. Finish your plan: write the pseudocode (or draw the flowchart) for your core loop below.

☐ My plan has a loop (every tick, or until Quit).

☐ My plan has at least one yes/no decision.

☐ My plan names the data: a list, a dictionary and a file.

Exercise 10.2: A fair task board

Write each function on its own sticky note. Fill in the table, then take turns choosing tasks: one each, like picking teams. Move notes from TO DO to DOING to DONE as you work.

Function name	What goes in	What comes out	Who	Done
`next_x`	x, direction	the new x		☐
				☐
				☐
				☐
				☐

Exercise 10.3: Build the core loop (M2)

Make a folder for your team’s project and save your program as `mygame.py` (or `mystudio.py`). Never call it `turtle.py`! Build it in small steps and press F5 after each one. For an arcade game,

the heart of the program is a loop that runs itself again:

the timer loop

```
def game_loop():
    if not state["running"]:
        return
    # TODO: move falling things, check for catches, add to the score
    screen.update()
    screen.ontimer(game_loop, 50)   # run again in 50 milliseconds
```

- ☐ My program runs without a red error.
- ☐ Something repeats by itself (it moves on the timer, or the menu comes back).
- ☐ At least one key or menu choice works.
- ☐ The start-up code is under `if __name__ == "__main__":`.
- ☐ Line 1 has a comment with our nicknames, and credit for any code we remixed.

Exercise 10.4: Partner code review

Swap seats with your partner (or another team). Run their program and read their code. Then write **two stars and a wish**: two things that are really good, and one kind idea. Be specific: "the name `next_x` tells me exactly what it does" helps more than "nice".

★

★

Wish:

Vocabulary box

pseudocode a plan for a program written in plain words, one step per line.

flowchart a drawing of a program: ovals for start and end, rectangles for steps, diamonds for yes/no questions.

decomposition breaking a big problem into small parts you can name and build one at a time.

core loop the part of a game or app that repeats over and over; the smallest version that works.

milestone a checkpoint on the way to the finished project, like M2 "core loop works".

code review reading someone else's code to help them find bugs and make it clearer.

Reflection

What was the smallest version of your project that worked today?

Was your task split fair? What would you change next week?

Week 11 Level 2: Intermediate

Project Studio 2

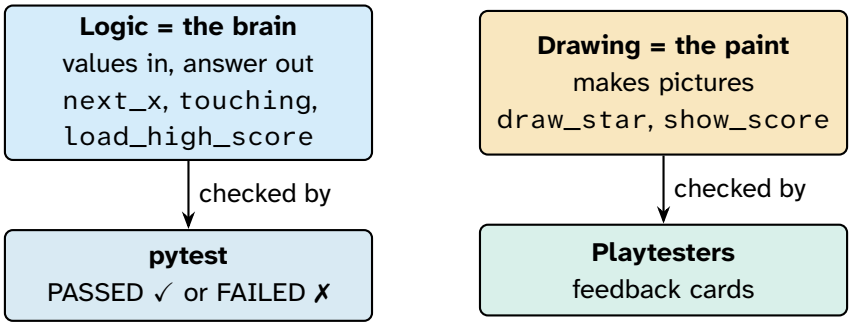
Tests check the brain of your program, playtesters check the fun, and polish makes it work for everyone.

What you will learn

- How to add data to your project: a list, a dictionary and a file that saves and loads (Milestone M3).
- How to write at least three pytest tests for your own functions.
- How to run a playtest and turn feedback into a bug list.
- How to check your project for accessibility, then polish it (Milestone M4).

The big idea: test the brain, playtest the fun

Your program has two kinds of functions. **Logic** functions (the brain) take values in and give an answer back with `return`: pytest can check those automatically. **Drawing** functions (the paint) make pictures: a computer cannot tell whether your star looks nice, but a playtester can.



Two tricks make your functions easy to test:

1. Put the start-up code (make the screen, start the loop) under `if __name__ == "__main__":`. Then pytest can borrow your functions without starting the game.
2. Give file functions the file name as a parameter. Then the game and the tests can use different files, for example `highscore.txt` for playing and `test_scores.txt` for testing, and your tests never wipe your real high score.

Test a **normal case** (an everyday value) and **edge cases** (values at the very limit: the edge of the screen, a score of `0`, a file that does not exist yet). Bugs love to hide at the edges.

Exercise 11.1: Add your data (M3)

Your project needs all three. Write where each one lives in your code.

Data	Its name in my code	What it remembers
a list		
a dictionary		
a file		

☐ I closed the program, ran it again, and the saved data came back.

☐ If the file does not exist yet, my program does not crash.

Exercise 11.2: Write three tests

Plan your tests first. Pick functions that `return` something.

Function	Input	Expected answer	Normal or edge?
`next_x`	`280, 1`	`280` (stays on screen)	edge

Make a new file in the same folder as your game. Its name must start with `test_`, for example `test_mygame.py`. Each test is a function whose name starts with `test_`:

test_mygame.py (part)

```
from mygame import load_high_score, next_x, save_high_score

def test_stops_at_the_right_edge():
    assert next_x(280, 1) == 280

def test_high_score_round_trip():
    save_high_score(42, "test_scores.txt")
    assert load_high_score("test_scores.txt") == 42
```

To run them: press F5 on your game once, close its window, then in Thonny choose **Tools > Open**

system shell... and type (use `python3` if `python` is not found):

Terminal

```
python -m pytest test_mygame.py -v
```

PASSED means the function gave the expected answer. **FAILED** means it did not: read the line starting with E to see what came out instead. A test that fails is not a disaster, it is a bug you found before a player did!

☐ I have at least 3 tests and they all say PASSED.

☐ I broke my code on purpose once, and a test turned to FAILED. (Then I fixed it back.)

Exercise 11.3: Playtest with feedback cards

Swap computers with another team (or ask a family member) and play their project for 4 minutes. Fill in a card for them. Be kind and specific.

Playtest feedback card

I liked...

I was confused when...

A bug I saw (what I did / what happened / what I expected):

One idea...

Back at your own project, turn the cards into a **bug list**. Mark each line MUST (it crashes or you cannot play), SHOULD (it is confusing) or COULD (a nice idea). Fix the MUST lines first.

What I did	What happened	What I expected	MUST / SHOULD / COULD

Exercise 11.4: Accessibility review and polish (M4)

Make your project work for as many people as possible. Check off each line or fix it.

- ☐ My colors come from the Okabe–Ito palette (`#E69F00`, `#56B4E9`, `#009E73`, `#0072B2`, `#D55E00`, `#CC79A7`).
- ☐ Nothing depends on color alone: good and bad things also have different shapes, sizes or labels.
- ☐ Text is big enough to read from a chair (font size 18 or more).
- ☐ The keys are explained on the screen, for example "← → to move".
- ☐ Nothing flashes fast (more than 3 times a second can hurt some people's eyes).
- ☐ There is an easy start: a slow first level or an easy setting.

One change I made:

Vocabulary box

playtest letting someone else use your program while you watch and take notes.

normal case a test with an everyday value, like moving from the middle of the screen.

edge case a test with a value at the very limit, like the edge of the screen, zero, or a missing file.

accessibility designing so that everyone can use it, including people who see colors differently or read more slowly.

polish small finishing touches that make a working program clearer and nicer to use.

bug list a list of problems found in testing, sorted so the most important ones get fixed first.

Reflection

Which test are you proudest of? What bug did it catch, or could it catch?

What did a playtester notice that you had not noticed?

Week 12 Level 2: Intermediate

Showcase and Celebration

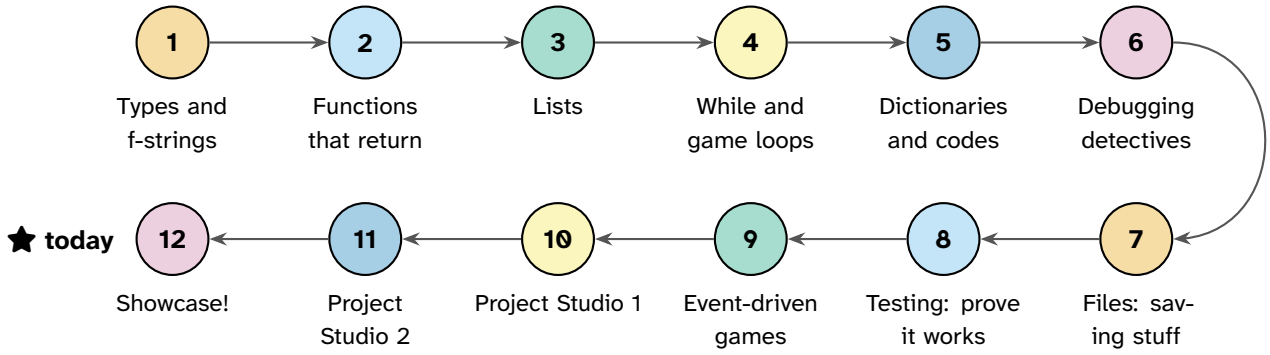
Share what you built, explain how it works, and see how far you have come.

What you will do today

- Present your project in about a minute: what it is, how one part works, and what you would add next.
- Play everyone's projects in a gallery walk and leave kind, specific feedback.
- Show what you learned on the exit assessment (it is not a test you can fail: it is your stats screen).
- Look back at your Level 2 journey, and peek at Level 3.

The big idea: look how far you have come

Twelve weeks ago you were sorting values into types. Now you have built a whole project with functions, lists, dictionaries, loops, files and tests. Here is the road you traveled:



How to explain your project

A great showcase pitch is short. Answer four questions, in this order:

1. **What is it?** "This is Comet Catch. You catch comets with a rocket."
2. **How does one part work?** Pick one function, the loop, the list, the dictionary or the file. "Our `next_x` function gets the position and the direction and returns the new position, but never off the screen. We have a test for that."

3. **One design decision.** Why did you choose something? “The comets are circles and the rocks are squares, so you can tell them apart even without color.”
4. **What next?** “Next we would add a slow mode for beginners.”

If something is not finished, that is normal. Show what works, then say what you would add next.

Exercise 12.1: My one-minute pitch

Fill in your presentation card, then practice once with your partner.
What is it?

How does one part work? (name a function, the loop, a list, a dictionary or the file)

One design decision and why:

What would we add next?

Exercise 12.2: Gallery walk feedback

Visit every project. Play it for 2–3 minutes, then leave a card: **two stars** (two specific things that are good) and **a wish** (one kind idea). Specific is kind: “the score text is big and easy to read” helps more than “cool”.

Feedback card for: _____

★

★

Wish:

Feedback card for: _____

★

★

Wish:

Exercise 12.3: My Level 2 skills check

Be honest: “not yet” just means “not yet”. Write a check mark in one column for each line.

I can…	Not yet	Getting there	I could teach it
use int, float, str and bool, and build messages with f-strings			
write a function with parameters that returns a value			
keep many things in a list and loop over it			
write a `while` loop that stops when it should			
look things up in a dictionary			
read a traceback and find the bug			
save data to a file and load it again			
write a pytest test and read PASSED / FAILED			
make a turtle game react to keys and a timer			
plan with pseudocode and share the work fairly			

Exercise 12.4: Looking ahead to Level 3

In Level 3 you will save your code with **git save points** (like save points in a video game), design your own **classes** (blueprints for game characters and objects), and build bigger projects. The example project is **Maze Muncher**: a maze game where a ghost hunts you down with a clever search algorithm. Three things I want to add to my project in a “version 2”:

Digital citizenship & safety: Sharing safely

Show your project to family and friends you know. Use your nickname on the screen and in high-score files. If someone takes a photo or video, film the screen, not faces. Never upload or post your project without deciding together with a parent.

Vocabulary box

showcase an event where makers show their work and explain it.

pitch a short talk that explains what your project is and why it is interesting.

design decision a choice you made on purpose, and the reason for it.

gallery walk visiting projects one by one, like in a museum, and leaving feedback.

version one stage of a project; version 2 is the next, improved one.

Reflection

What are you proudest of in your project?

What did you learn about working in a team?

What do you want to build next?